



Observabilidad y operación

18 de julio de 2026

Tabla de contenido

1. Introducción.....	3
2. Objetivos.....	4
3. Alcance	4
4. Arquitectura	5
5. Estrategia de logging	6
6. Formato de logs	7
7. Niveles de log.....	8
8. Datos que deben registrarse	9
9. Datos que no deben registrarse	10
10. Correlation ID.....	11
11. Idempotency key	12
12. Trazabilidad distribuida.....	13
13. Monitoreo de servicios.....	15
14. Métricas técnicas	15
15. Métricas de negocio	17
16. Dashboards.....	18
17. Manejo de errores.....	20
18. Reintentos	22
19. Dead Letter Queue.....	23
20. Limitaciones y oportunidades de mejora	25

1. Introducción

La observabilidad es un aspecto fundamental para garantizar el funcionamiento, seguimiento y mantenimiento de ECIEXPRESS. Debido a la naturaleza distribuida de su arquitectura, es necesario contar con mecanismos que permitan conocer el estado de los servicios, identificar errores, analizar el comportamiento de las solicitudes y reconstruir el recorrido de una operación a través de los distintos componentes.

ECIEXPRESS implementa la observabilidad de manera distribuida entre sus siete microservicios de negocio y el API Gateway. Cada componente genera registros estructurados sobre las solicitudes, respuestas, errores y eventos procesados, y envía información de telemetría a Azure Application Insights. Esta información puede ser consultada mediante KQL para apoyar el diagnóstico de fallos, el análisis de rendimiento y el seguimiento de la operación.

El sistema también cuenta con Reporting & Analytics Service como componente central de consulta y consolidación. Este servicio utiliza la información disponible de telemetría, monitoreo y eventos de negocio para construir indicadores técnicos y operativos que son presentados en el panel de monitoreo del frontend, disponible para los perfiles ADMIN y ANALYST.

La solución no utiliza actualmente una plataforma especializada de trazabilidad distribuida basada en herramientas como OpenTelemetry, Jaeger o Zipkin. En su lugar, la correlación entre solicitudes, eventos y registros se realiza mediante identificadores comunes, como X-Correlation-Id, requestId, eventId e idempotencyKey, que permiten relacionar la información generada por los diferentes servicios.

Además del logging y las métricas, la estrategia de observabilidad contempla mecanismos de monitoreo de salud, manejo estandarizado de errores, reintentos, control de tiempos de espera, seguimiento de eventos asíncronos y procedimientos básicos de recuperación ante fallos. Estos elementos permiten detectar comportamientos anómalos, reducir el tiempo de diagnóstico y disponer de evidencia sobre la operación real del sistema.

2. Objetivos

La estrategia de observabilidad de ECIEXPRESS busca proporcionar una visión clara y actualizada del estado técnico y operativo de la plataforma. Para ello, se apoya en logs estructurados, métricas, health checks, eventos de negocio e información de despliegue.

Sus objetivos principales son:

- Supervisar la disponibilidad del API Gateway y de los microservicios, con una meta mínima del 99 % durante el horario operativo del campus.
- Medir los tiempos de respuesta y verificar que las operaciones de consulta se mantengan por debajo de dos segundos en condiciones normales.
- Detectar y diagnosticar errores con información suficiente sobre el servicio, la operación y el contexto involucrado.
- Mantener trazabilidad entre solicitudes, logs y eventos mediante identificadores como X-Correlation-Id, requestId, eventId e idempotencyKey.
- Monitorear la publicación y el procesamiento de eventos en Azure Service Bus, incluyendo fallos, reintentos y mensajes pendientes.
- Consolidar métricas técnicas y de negocio en el panel de Reporting & Analytics para los perfiles ADMIN y ANALYST.
- Mantener actualizado el estado de los servicios y apoyar el diagnóstico y la recuperación ante fallos.

3. Alcance

El alcance incluye:

- Logging estructurado en el API Gateway y los microservicios.
- Correlación de solicitudes, eventos y registros mediante identificadores compartidos.
- Telemetría centralizada en Azure Application Insights.

- Health checks para verificar la disponibilidad y el estado de los servicios.
- Métricas técnicas, como latencia, errores y estado de los componentes.
- Métricas de negocio e indicadores operativos consolidados por Reporting & Analytics.
- Panel de monitoreo del frontend para los perfiles ADMIN y ANALYST.
- Manejo estandarizado de errores, reintentos y tiempos de espera.
- Seguimiento del backlog y de los mensajes no procesados en Azure Service Bus.
- Uso del patrón Transactional Outbox como mecanismo de confiabilidad y recuperación de eventos.
- Consulta del estado de los despliegues y del pipeline de CI/CD en Azure.

La solución de observabilidad se concentra en los mecanismos integrados directamente en la plataforma y en los servicios de Azure utilizados para su operación. El alcance actual no contempla una herramienta especializada de tracing distribuido basada en spans y traceId, ni dashboards externos como Grafana o plataformas de gestión de incidentes de terceros.

4. Arquitectura

La arquitectura se distribuye entre el API Gateway, los siete microservicios de negocio, Azure Application Insights, Azure Service Bus, Reporting & Analytics Service y el panel de monitoreo del frontend.

Cada componente del backend incorpora un StructuredLogger que genera registros en formato JSON y los envía a la salida estándar del contenedor. Estos registros incluyen información sobre solicitudes, respuestas, errores, eventos y contexto de ejecución. Adicionalmente, el logger remite trazas a Azure Application Insights mediante trackTrace.

Los servicios también inicializan el SDK de Application Insights mediante `setupAppInsights()`. Esta configuración permite recopilar automáticamente solicitudes HTTP, dependencias, excepciones y métricas de rendimiento. La información generada por ambos mecanismos se centraliza en Azure Application Insights y en el espacio de trabajo de Log Analytics asociado.

Reporting & Analytics Service funciona como componente central de consulta y consolidación. Sus responsabilidades dentro de la arquitectura de observabilidad incluyen:

- consultar la telemetría almacenada en Log Analytics mediante consultas KQL;

- ejecutar verificaciones HTTP sobre los endpoints /health del API Gateway y de los microservicios;
- consultar métricas de Azure Service Bus, como mensajes pendientes y mensajes enviados a la cola de mensajes muertos;
- obtener indicadores de negocio mediante consultas SQL de solo lectura sobre la información necesaria de pedidos y finanzas;
- consultar el estado de los flujos de CI/CD en GitHub Actions;
- y revisar el estado de los servicios desplegados en Azure Container Apps.

La información consolidada se expone mediante endpoints agrupados en:

- /kpis/*, para indicadores técnicos y de negocio;
- /monitoring/*, para salud, latencia y estado operativo;
- /deploy/*, para información de despliegue e infraestructura.

Estos endpoints se publican a través del API Gateway y son consumidos por el panel /admin/monitoring del frontend. El acceso está restringido a los roles ADMIN y ANALYST.

El panel actualiza la información de forma periódica: las verificaciones de salud se consultan aproximadamente cada cinco segundos y las métricas de latencia cada quince segundos

5. Estrategia de logging

El patrón principal se basa en un logger propio denominado StructuredLogger, implementado sobre la interfaz LoggerService de NestJS. Este logger genera registros en formato JSON y se encuentra configurado como mecanismo principal de logging en el API Gateway y en los servicios de Identity & Administration, Order & Communication, Financial, Fulfillment, Notifications y Reporting & Analytics.

Cada uno de estos componentes mantiene su propia implementación en `src/common/logger/structured.logger.ts` y la registra durante el arranque de la aplicación mediante `NestFactory.create(...)`. Los registros generados se envían a `stdout` y `stderr`, lo que permite su captura por la infraestructura de contenedores, y también pueden reenviarse a Azure Application Insights para su consulta centralizada.

Products Management Service utiliza una configuración diferente basada en Winston mediante nest-winston. En desarrollo, presenta los registros con un formato legible para consola, mientras que en producción utiliza salida JSON. Además, genera archivos separados para errores y registros generales con políticas de rotación, y dispone de un transporte personalizado que envía información a Application Insights. El nivel de logging puede configurarse mediante la variable de entorno LOG_LEVEL.

Pino se utiliza de forma puntual en algunos procesos internos de Identity & Administration relacionados con el Outbox y la publicación de eventos en Azure Service Bus. Su uso se limita a estos workers y no reemplaza el logger estructurado principal de las solicitudes HTTP.

De manera complementaria, existen algunos usos residuales de console.log y console.error durante el arranque del Gateway, principalmente antes de que el logger principal quede completamente inicializado.

En conjunto, la estrategia permite:

- Registrar la actividad de los servicios en un formato consistente y procesable
- Diferenciar mensajes informativos, advertencias y errores
- Relacionar los registros con solicitudes y eventos mediante identificadores de contexto
- Centralizar la consulta en Application Insights
- Conservar compatibilidad con el monitoreo nativo de contenedores y Azure

Aunque existen pequeñas variaciones entre servicios, el enfoque general mantiene una estructura homogénea basada en logs JSON, captura por consola y envío de telemetría a Azure Application Insights.

6. Formato de logs

Los servicios generan logs estructurados en formato JSON, lo que facilita su procesamiento, búsqueda y análisis tanto en la salida de los contenedores como en Azure Application Insights.

Cada registro puede incluir los siguientes campos:

- timestamp: fecha y hora en que se generó el registro.
- level: nivel del log, como debug, log, warn, error o fatal.

- **service:** nombre del servicio que produjo el registro.
- **context:** componente, clase o proceso desde el cual se originó el mensaje.
- **userId:** identificador del usuario relacionado con la operación, cuando aplica.
- **message:** descripción del evento, acción o error registrado.
- **trace:** información adicional de la excepción o traza de error, cuando corresponde.

Los niveles error y fatal se escriben en stderr, mientras que los niveles informativos y de advertencia se envían a stdout. Esta separación permite que la infraestructura de contenedores y las herramientas de monitoreo clasifiquen los registros según su severidad.

Cuando la variable `APPLICATIONINSIGHTS_CONNECTION_STRING` se encuentra configurada, cada log también se envía a Azure Application Insights mediante `trackTrace()`. La telemetría incluye dimensiones personalizadas como `serviceName` y `userId`, lo que permite filtrar y correlacionar los registros por servicio o usuario.

La telemetría remota es complementaria al funcionamiento de la aplicación. Si la conexión con Application Insights no está configurada o no se encuentra disponible, los servicios continúan operando y conservan la generación de logs en stdout y stderr.

7. Niveles de log

ECIEXPRESS utiliza distintos niveles de log para clasificar los registros según su propósito y severidad. Esta clasificación facilita la consulta de la telemetría, el diagnóstico de errores y la priorización de incidentes.

Los niveles definidos son:

- **verbose:** registra información detallada útil para el análisis interno de una operación.
- **debug:** almacena datos de apoyo para tareas de desarrollo, pruebas y diagnóstico técnico.
- **info:** registra operaciones normales del sistema, como inicio de servicios, procesamiento exitoso de solicitudes y ejecución de procesos esperados. Este nivel corresponde al método `log()` de NestJS.
- **warn:** identifica situaciones anómalas que no interrumpen la operación, pero requieren seguimiento.

- **error:** registra fallos que afectan una solicitud, evento o proceso específico.
- **fatal:** representa errores críticos que pueden impedir el arranque o comprometer la continuidad de un servicio.

Al enviarse a Azure Application Insights, estos niveles se relacionan con sus categorías de severidad de la siguiente manera:

Nivel	Severidad en Application Insights
verbose	verbose
debug	verbose
info	Information
warn	Warning
error	Error
fatalx	Critical

Esta equivalencia permite consultar y filtrar los registros de manera consistente tanto en los contenedores como en Application Insights.

8. Datos que deben registrarse

Los logs deben incluir la información necesaria para identificar el origen de cada registro, relacionarlo con una solicitud o evento y facilitar el diagnóstico de errores.

Los datos principales son:

- **timestamp:** fecha y hora en que se generó el registro.
- **level:** nivel de severidad del log.
- **service:** nombre del microservicio que produjo el registro, definido mediante la constante `SERVICE_NAME`.
- **message:** descripción de la operación, evento o error registrado.
- **userId:** identificador del usuario relacionado con la solicitud, cuando aplica.

El `userId` se obtiene mediante un contexto basado en `AsyncLocalStorage`. Este contexto es inicializado por un middleware que lee el encabezado `x-user-id` de cada solicitud y permite que el identificador esté disponible durante el procesamiento completo del request.

En los procesos de publicación y consumo de eventos también se registran datos específicos de trazabilidad:

- **correlationId:** identificador utilizado para relacionar una operación entre solicitudes, logs y eventos.
- **eventId:** identificador interno del evento almacenado en la tabla `outbox_events`.
- **routingKey:** clave utilizada para clasificar o dirigir el evento.
- **eventType:** tipo de evento de negocio publicado.

En el manejo de excepciones, especialmente en Fulfillment Service, los errores con estado HTTP igual o superior a 500 incluyen además el `correlationId` y el objeto completo de la excepción. Esto permite conservar el detalle técnico necesario para analizar fallos internos sin limitar el diagnóstico al mensaje mostrado al cliente.

9. Datos que no deben registrarse

La estrategia de logging debe evitar el almacenamiento de información sensible o innecesaria para el diagnóstico técnico. Como principio general, los logs deben contener únicamente los datos indispensables para identificar una operación, relacionarla con un usuario o evento y analizar su resultado.

No deben registrarse:

- Contraseñas ni credenciales de autenticación.
- Tokens de Firebase, JWT, claves de sesión o encabezados de autorización.
- Claves privadas, secretos, cadenas de conexión o credenciales de servicios externos.
- Datos financieros sensibles, como información completa de tarjetas o cuentas bancarias.
- Contenido completo de webhooks o solicitudes cuando incluya información sensible.
- Información personal identificable distinta de los identificadores técnicos necesarios, como `userId`, `orderId` o `storeId`.
- Códigos de retiro completos cuando su exposición pueda permitir una entrega no autorizada.
- Trazas técnicas completas en las respuestas enviadas al cliente.

Cuando ocurre una excepción, el detalle técnico y el stack trace pueden registrarse internamente para fines de diagnóstico, especialmente en errores de servidor. Sin embargo, esta información no debe incluirse en la respuesta HTTP. El cliente debe recibir únicamente un mensaje controlado, el código de estado correspondiente y, cuando aplique, un identificador de correlación que permita localizar el incidente en los logs.

El manejo de registros sigue el principio de minimizar la exposición de datos personales: se utilizan identificadores internos para mantener la trazabilidad sin almacenar información adicional del usuario. De esta forma, los logs conservan su utilidad operativa sin convertirse en una fuente innecesaria de datos sensibles.

10. Correlation ID

ECIEXPRESS utiliza el encabezado X-Correlation-Id para relacionar las solicitudes, respuestas, logs y eventos que forman parte de una misma operación distribuida. Este identificador permite seguir el recorrido de una petición desde su ingreso por el API Gateway hasta los microservicios y procesos asíncronos involucrados.

El API Gateway actúa como punto principal de generación y propagación. Cuando una solicitud incluye X-Correlation-Id, conserva su valor; en caso contrario, genera uno nuevo mediante UUID y lo inyecta antes de reenviar la petición al servicio correspondiente. Este comportamiento aplica tanto a rutas autenticadas como públicas.

En Identity & Administration Service y Fulfillment Service, interceptores propios reciben el identificador, lo almacenan en el contexto de la solicitud y lo incluyen nuevamente en la respuesta HTTP. Esto permite que el cliente y los servicios internos mantengan una referencia común durante todo el flujo.

Order & Communication, Products Management y Financial Service no generan un identificador adicional cuando la solicitud ya proviene del Gateway. Estos servicios reutilizan el valor recibido y lo incorporan en los eventos que publican, manteniendo la continuidad entre la comunicación síncrona y asíncrona.

En los eventos de negocio, el correlationId forma parte del sobre del mensaje y se conserva durante la publicación y el consumo en Azure Service Bus. De esta

manera, una misma operación puede correlacionarse aunque atraviere varios microservicios o continúe mediante procesos desacoplados.

El flujo general es el siguiente:

1. El cliente envía una solicitud, con o sin X-Correlation-Id.
2. El Gateway conserva el valor recibido o genera uno nuevo.
3. El identificador se propaga a los microservicios mediante el encabezado HTTP.
4. Los servicios lo incluyen en sus logs y eventos.
5. Los consumidores reutilizan el mismo valor al continuar el procesamiento.
6. El identificador puede consultarse en Application Insights para reconstruir el flujo completo.

Aunque la implementación se distribuye entre varios componentes, el API Gateway garantiza que toda solicitud que ingresa a la plataforma cuenta con un identificador de correlación antes de llegar a los servicios internos.

11. Idempotency key

ECIEXPRESS utiliza Idempotency-Key para evitar que una misma solicitud o evento produzca efectos duplicados cuando es reenviado o procesado más de una vez. Este mecanismo es especialmente importante en operaciones distribuidas como pagos, cambios de estado, generación de códigos de retiro y publicación de eventos.

En la comunicación asíncrona, los servicios publicadores incorporan un sobre estándar que incluye:

- idempotencyKey
- occurredAt
- source
- eventVersion
- correlationId
- payload

Cuando la operación de origen ya cuenta con una clave de idempotencia, esta se conserva en el evento. Si no existe, el publicador genera una nueva mediante

UUID. En Identity & Administration Service, la clave también se almacena en la columna `idempotencyKey` de la tabla `outbox_events`.

En las operaciones HTTP, el encabezado `Idempotency-Key` se encuentra habilitado por el API Gateway y puede utilizarse en mutaciones que requieran protección frente a reintentos, como confirmaciones, entregas o transacciones.

Para evitar duplicados durante el consumo de eventos, los servicios registran las claves procesadas en una tabla `processed_events`. Antes de aplicar cualquier cambio de negocio, el consumidor verifica si la clave ya existe:

1. Si la clave no ha sido procesada, el servicio registra el evento y ejecuta la operación.
2. Si la clave ya existe, el evento se reconoce como duplicado y no vuelve a aplicar sus efectos.

En Order & Communication Service, cuando un evento no incluye `idempotencyKey`, se construye una clave alternativa a partir de la ruta del evento y el identificador del pedido, siguiendo el formato:

`{routingKey}:{orderId}`

Este enfoque permite que la recepción repetida de un evento no genere cobros dobles, códigos de retiro duplicados, entregas repetidas ni transiciones de estado aplicadas más de una vez.

12. Trazabilidad distribuida

ECIEXPRESS implementa la trazabilidad distribuida mediante Azure Application Insights y la propagación de identificadores comunes entre solicitudes, logs y eventos.

Los ocho componentes backend inicializan `setupAppInsights()` antes de crear la aplicación NestJS. Esta configuración habilita la recolección automática de:

- solicitudes HTTP;
- dependencias;
- excepciones;
- métricas de rendimiento;
- y métricas en tiempo real.

La captura automática de consola permanece desactivada para evitar duplicar los registros que StructuredLogger ya envía explícitamente. Además, el SDK utiliza caché en disco para conservar temporalmente la telemetría cuando ocurre un fallo de envío.

Cuando APPLICATIONINSIGHTS_CONNECTION_STRING está configurada, cada servicio envía su telemetría a Application Insights. En entornos donde esta variable no está disponible, la aplicación continúa operando sin interrumpir el arranque.

Cada componente identifica su telemetría mediante cloud_RoleName, utilizando el valor definido en SERVICE_NAME. Esto permite distinguir los servicios en Application Insights, consultar la información por AppRoleName y visualizar sus relaciones en el mapa de aplicaciones de Azure.

Reporting & Analytics Service consulta la telemetría almacenada en el workspace de Log Analytics mediante KQL y autenticación con DefaultAzureCredential. Las consultas se realizan principalmente sobre:

- AppRequests, para solicitudes y tiempos de respuesta;
- AppExceptions, para errores y excepciones;
- AppTraces, para logs y seguimiento de eventos.

La trazabilidad de los eventos entre microservicios se construye combinando el correlationId, los metadatos del evento y los mensajes estructurados registrados durante su publicación y consumo. Reporting & Analytics identifica patrones como:

- evento publicado;
- evento recibido;
- error durante la publicación;
- evento marcado como fallido.

Los mensajes enviados a la cola de mensajes muertos no se obtienen desde los logs de Application Insights. Su estado se consulta directamente mediante las métricas de Azure Service Bus, ya que corresponden a información propia de la infraestructura de mensajería.

13. Monitoreo de servicios

ECIEXPRESS realiza monitoreo activo sobre el API Gateway y los siete microservicios de negocio mediante Reporting & Analytics Service. Este componente consulta periódicamente el endpoint GET /health de cada servicio para verificar su disponibilidad y medir el tiempo de respuesta.

Las direcciones de los servicios se configuran mediante variables de entorno con el formato SERVICE_<NOMBRE>_URL. Cada verificación utiliza un tiempo máximo de espera de cuatro segundos. Como resultado, el sistema registra para cada componente:

- estado de disponibilidad (up);
- tiempo de respuesta en milisegundos (responseMs);
- código de estado HTTP (statusCode);
- fecha y hora de la verificación (checkedAt).

La información consolidada se expone mediante el endpoint GET /kpis/health, protegido para los roles ADMIN y ANALYST. El panel de monitoreo del frontend consulta este endpoint cada cinco segundos, lo que permite mantener una vista actualizada del estado operativo de la plataforma.

Notifications Service incorpora además un mecanismo interno de activación preventiva para reducir el impacto de posibles periodos de inactividad en la infraestructura de hosting. Este proceso realiza solicitudes a los endpoints de salud de los microservicios y genera un reporte con el resultado de las verificaciones, que puede enviarse al correo configurado en WAKEUP_REPORT_EMAIL.

El endpoint utilizado para esta operación permanece bloqueado para accesos externos desde el API Gateway y solo puede ejecutarse dentro del entorno interno autorizado.

14. Métricas técnicas

Reporting & Analytics Service consolida las métricas técnicas de ECIEXPRESS mediante consultas KQL sobre la telemetría almacenada en Azure Application Insights y Log Analytics. Esta información se expone a través de los endpoints

del módulo de KPIs y permite analizar el volumen de solicitudes, la latencia, los errores y el comportamiento de los eventos distribuidos.

Resumen general

El endpoint GET /kpis/overview presenta una visión consolidada del comportamiento de las solicitudes registradas en AppRequests. Incluye:

- número total de solicitudes;
- número de solicitudes fallidas;
- percentil 50 de duración (p50Ms);
- percentil 95 de duración (p95Ms);
- tasa de error;
- y distribución de resultados por servicio.

El mismo análisis se realiza por AppRoleName, lo que permite comparar el rendimiento y la tasa de fallos entre los distintos componentes.

Latencia

El endpoint GET /kpis/latency genera una serie temporal agrupada por minuto y por servicio. Para cada intervalo presenta:

- tiempo promedio de respuesta;
- percentil 95;
- percentil 99;
- y cantidad de solicitudes procesadas.

Estas métricas permiten identificar degradaciones, picos de demanda y servicios con tiempos de respuesta superiores a los esperados.

Excepciones

El resumen técnico incluye las excepciones más frecuentes registradas en AppExceptions. La consulta agrupa la información por tipo de excepción y servicio, y presenta una muestra del mensaje asociado.

Esto facilita la identificación de errores repetitivos y de los componentes en los que se concentran.

Métricas de eventos

El endpoint GET /kpis/events consolida información sobre los eventos publicados, recibidos y fallidos por servicio y clave de enrutamiento.

El cálculo se realiza a partir de los registros almacenados en AppTraces, identificando los mensajes generados durante la publicación y consumo de eventos.

Rango de consulta y resiliencia

Las métricas pueden consultarse dentro de un rango configurable, expresado en horas o minutos. Los límites aceptados son:

- entre 1 y 720 horas;
- entre 1 y 1440 minutos.

Las distintas secciones del resumen se ejecutan de manera independiente mediante Promise.allSettled. De esta forma, si una consulta falla, las demás pueden continuar respondiendo y el panel conserva la información disponible.

Seguridad de las consultas

Las consultas KQL utilizadas por Reporting & Analytics están definidas como constantes previamente autorizadas. Los valores enviados por el usuario no se interpolan directamente en las expresiones KQL.

15. Métricas de negocio

Reporting & Analytics Service consolida indicadores de negocio a partir de consultas SQL de solo lectura sobre la información de pedidos y finanzas. Estas consultas se ejecutan mediante conexiones independientes, con un tiempo máximo de espera de ocho segundos y un límite reducido de conexiones para evitar afectar la operación de los servicios consultados.

Las métricas se encuentran definidas en un catálogo cerrado de consultas autorizadas. Los parámetros se envían de forma enlazada y se validan según su tipo, como uuid, int o bigint, evitando la construcción dinámica de sentencias SQL.

El catálogo incluye los siguientes indicadores:

1. **Pedidos por tienda:** cantidad de pedidos y monto total procesado por cada punto de venta.
2. **Producto más vendido por tienda:** producto con mayor número de unidades vendidas.
3. **Producto menos vendido por tienda:** producto con menor número de unidades vendidas dentro del periodo consultado.
4. **Usuarios con mayores recargas:** compradores que acumulan el mayor valor de recargas aprobadas por encima de un umbral definido.
5. **Pagos fallidos:** transacciones rechazadas agrupadas según la causa del fallo.
6. **Pedidos atascados:** pedidos que permanecen en los estados PENDING_PAYMENT o PAYMENT_APPROVED durante más tiempo del permitido.
7. **Reembolsos:** cantidad y valor de las transacciones reembolsadas de forma total o parcial.
8. **Compras en hora pico por tienda:** operaciones realizadas dentro de los periodos de mayor demanda configurados para cada punto de venta.

La disponibilidad de estas consultas puede consultarse mediante GET /kpis/quick, mientras que cada indicador se ejecuta a través de GET /kpis/quick/:key.

16. Dashboards

ECIEXPRESS cuenta con un dashboard propio de observabilidad integrado en el frontend, disponible en la ruta /admin/monitoring para los perfiles ADMIN y ANALYST.

El dashboard consume la información exclusivamente a través del API Gateway, utilizando el token de Firebase y el identificador de sesión correspondiente. De esta manera, mantiene el mismo esquema de autenticación y control de acceso definido para el resto de la plataforma.

El Centro de Monitoreo se organiza en las siguientes secciones:

Salud de los microservicios

Presenta una tarjeta por servicio con información sobre:

- estado activo o caído;
- latencia promedio;
- cantidad de solicitudes por minuto;

- y última actualización disponible.

La información de salud se actualiza cada cinco segundos, mientras que las métricas de latencia se consultan cada quince segundos. Al seleccionar un servicio, se abre una vista detallada con su comportamiento histórico.

Flujo de eventos

Muestra la cantidad de eventos:

- generados;
- recibidos;
- y fallidos.

La información se agrupa por servicio y clave de enrutamiento, y puede consultarse dentro de una ventana de tiempo configurable, con un valor predeterminado de sesenta minutos.

Estado de Azure Service Bus

Presenta una tabla con la cantidad de mensajes activos y mensajes enviados a dead-letter por entidad o suscripción.

Esta sección se encuentra disponible en el entorno desplegado, donde Reporting & Analytics puede consultar directamente las métricas de Azure Service Bus.

Métricas de negocio y consultas rápidas

Incluye tablas con indicadores como pedidos por tienda, productos más y menos vendidos, pagos fallidos, reembolsos, pedidos atascados y compras en hora pico.

Las consultas que requieren parámetros permiten seleccionarlos desde la interfaz. Para los indicadores asociados a tiendas, el sistema incluye un selector visual mediante StoreMapModal.

Latencia histórica por servicio

Al seleccionar un microservicio se presenta una vista detallada con:

- promedio de latencia;
- percentil 95;

- percentil 99;
- solicitudes por minuto;
- y comportamiento dentro de rangos de 10, 30 o 60 minutos.

La gráfica se construye mediante componentes SVG propios del frontend.

Estado de despliegue

La vista detallada de cada servicio incluye información sobre su estado de despliegue, permitiendo relacionar cambios recientes en CI/CD con variaciones de latencia, errores o disponibilidad.

Comportamiento ante información no disponible

El dashboard maneja de forma controlada los entornos donde Application Insights o Azure Monitor no están configurados. En estos casos, las secciones dependientes de dichos servicios muestran un mensaje indicando que la información está disponible únicamente en el entorno desplegado, sin afectar el funcionamiento general de la página.

17. Manejo de errores

ECIEXPRESS utiliza mecanismos de manejo de errores adaptados a las necesidades de cada microservicio. Aunque no existe un único filtro compartido por toda la plataforma, los servicios aplican principios comunes: devolver respuestas controladas, conservar un identificador de correlación y evitar exponer información técnica sensible al cliente.

Identity & Administration Service

Identity & Administration utiliza un filtro global de excepciones que normaliza las respuestas de error con la siguiente información:

- código de estado HTTP;
- mensaje;
- correlationId;
- fecha y hora;
- ruta solicitada.

Este formato permite que el cliente identifique el tipo de error y utilice el identificador de correlación para localizar el incidente en los logs.

Fulfillment Service

Fulfillment utiliza un filtro de excepciones orientado a errores de dominio. Las respuestas incluyen:

- código del error;
- mensaje comprensible para el usuario;
- correlationId;
- y detalles adicionales cuando corresponde.

El servicio diferencia entre errores de negocio previamente tipificados, errores de validación de entrada y fallos internos. Los mensajes técnicos se transforman en respuestas controladas y en español, mientras que el detalle interno se conserva únicamente en los logs.

Cuando el error tiene un estado igual o superior a 500, se registra internamente la excepción completa para facilitar el diagnóstico. El stack trace nunca se envía al cliente.

Order & Communication Service

Order & Communication utiliza un filtro global para capturar excepciones no controladas y devolver respuestas consistentes desde el servicio.

Este mecanismo evita que los errores internos se propaguen directamente al cliente y permite registrar el contexto necesario para su análisis.

Manejo estándar en los demás componentes

Products Management, Financial, Notifications, Reporting & Analytics y el API Gateway utilizan principalmente el mecanismo de manejo de errores proporcionado por NestJS y Express.

En estos componentes, las excepciones HTTP y las validaciones se transforman en respuestas con el código de estado correspondiente, mientras que los fallos no controlados se registran mediante el logger configurado en cada servicio.

Principios generales

El manejo de errores de ECIEXPRESS sigue los siguientes principios:

- No exponer stack traces ni detalles internos al cliente.
- Incluir un correlationId cuando esté disponible.
- Utilizar códigos HTTP coherentes con la causa del error.
- Presentar mensajes comprensibles para los usuarios.
- Registrar internamente el detalle técnico necesario para el diagnóstico.
- Diferenciar errores de validación, autorización, negocio e infraestructura.
- Evitar que una excepción no controlada revele credenciales, tokens, datos personales o información sensible.

18. Reintentos

ECIEXPRESS aplica mecanismos de reintento principalmente en la publicación de eventos asíncronos, con el propósito de reducir la pérdida de mensajes ante fallos temporales de Azure Service Bus.

Identity & Administration Service

Identity & Administration utiliza el patrón Transactional Outbox con un máximo de cinco intentos de publicación.

Cuando un intento falla, el evento permanece almacenado y se programa un nuevo intento mediante backoff exponencial, calculado con base en el número de reintentos realizados. Una vez alcanzado el límite, el evento cambia al estado FAILED para su posterior revisión.

Fulfillment Service

Fulfillment aplica el mismo enfoque de reintentos con backoff exponencial. Los eventos pendientes permanecen en el Outbox y se vuelven a publicar después de un intervalo creciente.

La cantidad máxima de intentos se configura mediante la variable de entorno OUTBOX_MAX_RETRIES, lo que permite ajustar el comportamiento según el entorno de despliegue.

Order & Communication Service

Order & Communication también utiliza un mecanismo de reintento para los eventos almacenados en el Outbox.

En este servicio, el contador de intentos se incrementa hasta un máximo de cinco y los eventos pendientes se vuelven a procesar en el siguiente ciclo del worker, ejecutado periódicamente. Este mecanismo utiliza intervalos constantes de consulta, en lugar de un tiempo de espera exponencial individual por evento.

Products Management y Financial Service

Products Management y Financial Service realizan la publicación de eventos directamente en Azure Service Bus. Si se presenta un fallo temporal, la operación se registra en los logs y el flujo principal de negocio continúa.

En estos componentes, la recuperación de una publicación fallida depende de los mecanismos de trazabilidad y reconciliación definidos para la operación, por lo que los errores quedan registrados para su identificación y tratamiento posterior.

Comportamiento general

La estrategia de reintentos sigue estos principios:

- Los fallos temporales de mensajería no deben provocar efectos duplicados.
- Los servicios con Transactional Outbox conservan los eventos pendientes hasta completar su publicación o agotar el número máximo de intentos.
- Cada intento fallido debe quedar registrado con su causa y contador correspondiente.
- Los eventos que alcanzan el máximo de reintentos deben quedar identificados para revisión.
- La idempotencia de los consumidores evita que un evento reenviado produzca cambios repetidos.

19. Dead Letter Queue

ECIEXPRESS utiliza la funcionalidad nativa de **Dead Letter Queue de Azure Service Bus** para aislar los mensajes que no pueden procesarse correctamente después de varios intentos.

La configuración del bus define el tópico `eciexpress_events` y cinco suscripciones asociadas a Notifications, Financial, Products, Orders y Fulfillment. Cada suscripción utiliza las siguientes condiciones principales:

- duración del bloqueo del mensaje de un minuto;
- procesamiento sin sesiones;
- máximo de diez intentos de entrega;
- tiempo de vida predeterminado de una hora para los mensajes;
- y subcola nativa de dead-letter para mensajes que superan el número máximo de entregas.

Cuando un consumidor no logra procesar un mensaje y se alcanza `MaxDeliveryCount = 10`, Azure Service Bus lo mueve automáticamente a la subcola de dead-letter de la suscripción correspondiente. Esto evita que el mismo mensaje continúe reintentándose de forma indefinida y permite su revisión posterior.

Los mensajes que expiran por superar su tiempo de vida no se envían automáticamente al dead-letter por esa causa, debido a que la opción `DeadLetteringOnMessageExpiration` se encuentra desactivada.

Monitoreo del backlog y mensajes en dead-letter

Reporting & Analytics Service consulta las métricas nativas de Azure Service Bus mediante Azure Monitor. Entre las métricas supervisadas se encuentran:

- `ActiveMessages`, para conocer el volumen de mensajes pendientes;
- `DeadletteredMessages`, para identificar mensajes enviados a dead-letter.

La información se consulta por entidad o suscripción, con intervalos de cinco minutos, y se expone mediante el endpoint `GET /kpis/servicebus`.

El dashboard presenta estos datos en la sección **Service Bus · backlog**, permitiendo que los perfiles ADMIN y ANALYST identifiquen acumulaciones de mensajes o fallos persistentes de procesamiento.

Disponibilidad de la información

La consulta de estas métricas se encuentra habilitada en el entorno desplegado y requiere:

- `SERVICEBUS_RESOURCE_ID`;
- `AZURE_METRICS_ENDPOINT`;
- y permisos de lectura sobre Azure Monitor, como el rol Monitoring Reader.

En entornos locales donde estas configuraciones no están disponibles, el servicio responde de forma controlada indicando que la información del bus no puede consultarse.

20. Limitaciones y oportunidades de mejora

La estrategia de observabilidad implementada permite supervisar la disponibilidad, el rendimiento, los errores, los eventos y los despliegues de ECIEXPRESS. Sin embargo, se identifican oportunidades de evolución que permitirían fortalecer la trazabilidad, la recuperación ante fallos y la uniformidad operativa de la plataforma.

Trazabilidad distribuida

La correlación entre servicios se realiza mediante X-Correlation-Id, identificadores de eventos y análisis de logs en Application Insights. Como evolución, puede incorporarse una solución de tracing distribuido basada en spans y trazas estructuradas, que permita visualizar con mayor precisión el recorrido y la duración de una operación entre microservicios.

Estandarización del manejo de errores

Los servicios aplican mecanismos de control de excepciones, pero no todos utilizan el mismo formato de respuesta. Se plantea como mejora definir un contrato transversal de errores que unifique campos como código, mensaje, correlationId, fecha, ruta y detalles permitidos.

Confiabilidad de eventos

Identity & Administration, Order & Communication y Fulfillment utilizan mecanismos de persistencia y reintento mediante Transactional Outbox. Products Management y Financial publican algunos eventos directamente en Azure Service Bus, por lo que pueden fortalecerse mediante la adopción del mismo patrón o de un proceso automático de reconciliación.

Alertas automáticas

El panel permite consultar el estado de la plataforma, pero el seguimiento depende actualmente de la revisión por parte de administradores o analistas. Como mejora, pueden configurarse alertas automáticas ante situaciones como:

- caída de un servicio;

- aumento de la tasa de errores;
- crecimiento del backlog;
- aparición de mensajes en dead-letter;
- o latencia superior a los umbrales definidos.

Control de tiempos de espera

Las verificaciones realizadas por Reporting & Analytics cuentan con tiempos de espera definidos. Sin embargo, puede incorporarse una política uniforme de timeout en las solicitudes del API Gateway hacia los microservicios para evitar esperas prolongadas cuando un componente no responde.

Dependencia del entorno desplegado

Las métricas de Application Insights, Azure Service Bus y Azure Container Apps se encuentran disponibles en el entorno desplegado. En ejecución local, el sistema continúa funcionando y presenta estas secciones como no disponibles, sin afectar las funcionalidades principales.

Evolución del dashboard

ECIEXPRESS utiliza un dashboard propio integrado en el frontend y conectado con los servicios de Azure. Este enfoque centraliza el acceso dentro de la plataforma y evita incorporar herramientas externas adicionales para la visualización actual. Como evolución, el dashboard puede complementarse con nuevas gráficas, umbrales, alertas y vistas históricas sin modificar el flujo de acceso definido para los roles ADMIN y ANALYST.